



Cat Monitor

Autonomous AI Agent — Project Notes

March 2026 · Walter P · spicysnickers

Project Overview

Cat Monitor is a fully local, autonomous AI agent that processes motion-triggered security camera footage from a backyard stray cat feeding station. It identifies individual cats across visits, tracks their health over time, and reasons about what it observes — without any cloud services or human intervention.

Hardware	AMD Ryzen 9 9900 · 64GB RAM · 1TB NVMe · 8TB SATA · 24TB External
GPU	AMD Radeon AI Pro R9700 · 32GB VRAM
OS	Ubuntu 24.04 LTS
Vision Model	Qwen3-VL-8B-Instruct served via vLLM on ROCm
Cameras	Reolink · Platform Front + Platform Right · Motion triggered FTP upload
Cloud services	None — fully local inference, no API costs

System Architecture

The system is structured in layers. Each layer has a single responsibility and can be developed or replaced independently.

Layer Stack

✓	Reolink Cameras	Hardware
✓	FTP Drop Folder	Local network
✓	Folder Watcher	watcher.py
✓	Frame Extractor	tools/frames.py
✓	Vision Model API	vLLM + Qwen3-VL
✓	Cat Identity Engine	db/store.py + ChromaDB
✓	Visit Logger	db/store.py + SQLite
✓	Agent Loop	agent/loop.py
○	Reflective Agent	Planned — needs data
○	Dashboard	Planned

Data Flow

- Reolink camera detects motion → records clip → uploads via FTP
- watcher.py detects new file → waits for upload to complete → triggers agent
- Agent calls `extract_best_frames()` → FFmpeg extracts 1fps frames → Qwen3-VL scores each frame → top 4 selected
- Agent calls `analyze_cat_image()` on each best frame → structured JSON description returned
- Agent calls `process_visit()` → ChromaDB identity match → SQLite visit logged
- Agent produces natural language summary → logged

Project Structure

```
~/catmonitor/
├── agent/
│   └── loop.py           # Agent loop - core reasoning
├── tools/
│   ├── vision.py        # analyze_cat_image() tool
│   └── frames.py         # extract_best_frames() tool
```

```
├─ db/
│   └─ store.py          # SQLite + ChromaDB identity engine
├─ data/
│   ├── catmonitor.db    # SQLite – visits, cats, alerts
│   └─ chromadb/         # Vector embeddings – cat identity
├─ watcher.py           # Folder watcher – new clips
├─ process_existing.py   # Batch process existing clips
└─ catmonitor.log        # Running log
```

Components

1. vLLM Vision Model Server

Qwen3-VL-8B-Instruct served via vLLM in a ROCm Docker container. Exposes an OpenAI-compatible REST API on localhost:8000. All vision analysis and agent reasoning calls go through this endpoint.

Docker run command:

```
docker run --rm \
  --group-add=video \
  --cap-add=SYS_PTRACE \
  --security-opt seccomp=unconfined \
  --device /dev/kfd \
  --device /dev/dri \
  -p 8000:8000 \
  --ipc=host \
  -e "HF_TOKEN=$HF_TOKEN" \
  -e VLLM_ROCM_USE_AITER=1 \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  vllm/vllm-openai-rocm:v0.14.0 \
  --model Qwen/Qwen3-VL-8B-Instruct \
  --max-model-len 16384 \
  --enable-auto-tool-choice \
  --tool-call-parser hermes
```

Key flags: `--max-model-len 16384` prevents KV cache overflow. `--enable-auto-tool-choice` and `--tool-call-parser hermes` enable the agent tool use loop. Without these the agent cannot call tools.

2. Vision Tool — tools/vision.py

Sends a single image frame to the vLLM endpoint and returns a structured JSON description of any cats present. This is the agent's primary sense — what it uses to see.

Key design decisions:

- Structured output with controlled vocabularies — `body_condition` is `poor/fair/good/excellent`, not free text
- Camera name read from overlay text in the image itself — no need to pass it separately
- Validation rules in the prompt prevent non-cats (birds, raccoons) being classified as cats
- `health_flags` and `distinctive_markings` are arrays — empty if nothing notable
- `confidence` field set to low if lighting is poor or cat partially obscured

Output schema:

```
{
  "cat_present": boolean,
  "cat_count": integer,
  "cats": [{
    "coat_color": string,
    "coat_pattern": solid|tabby|bicolor|tortoiseshell|calico|unknown,
    "coat_length": short|medium|long|unknown,
    "size": small|medium|large|unknown,
    "build": lean|normal|stocky|unknown,
    "distinctive_markings": [array of strings],
    "eye_color": yellow|green|blue|amber|unknown,
    "body_condition": poor|fair|good|excellent,
    "behavior": eating|resting|alert|grooming|fleeing|fighting|unknown,
    "health_flags": [array of strings],
    "confidence": low|medium|high
  }],
  "camera": platform_front|platform_right|unknown,
  "lighting": good|partial|poor,
  "notes": string
}
```

3. Frame Extractor — `tools/frames.py`

Extracts the best frames from a video clip using an async pipeline. Rather than sampling frames blindly, it uses the vision model itself to score each frame for quality before selecting the top 4.

Pipeline:

- FFmpeg extracts 1 frame per second and resizes to 640px wide in one pass
- All frames scored concurrently with asyncio (`MAX_CONCURRENT=1` for single vLLM instance)
- Each frame scored 1-10: cat visibility, sharpness, angle, lighting

- Top 4 selected with minimum 3-second spacing for temporal diversity
- Returns list of image paths for the agent to analyze

Important: 4K Reolink frames consume ~8000 tokens at full resolution — the entire context window. Resizing to 640px wide drops this to ~238 tokens. Always resize before sending to the model.

4. Cat Identity Engine — db/store.py

The core of autonomous cat identification. Uses two databases working together — SQLite for structured facts and ChromaDB for vector-based identity matching.

SQLite Schema

```
cats      - cat_id, first_seen, last_seen, visit_count, description
visits    - visit_id, cat_id, timestamp, camera, clip_path,
            behavior, body_condition, health_flags, lighting,
            confidence, notes, raw_json
alerts    - alert_id, cat_id, timestamp, alert_type, detail, resolved
```

ChromaDB Identity Matching

Each cat is stored as a vector embedding of their description text. When a new cat arrives, their description is embedded and compared against all known profiles.

Description text format (embedded for identity):

```
"color:orange color:orange pattern:solid eyes:yellow eyes:yellow
coat:short size:medium build:normal markings:none"
```

Color and eye color are repeated to give them stronger weight in the embedding. They are also used as hard metadata filters — a black cat with green eyes will never match an orange cat with yellow eyes regardless of vector distance.

Matching Logic

- Hard filter: coat_color AND eye_color must match exactly in ChromaDB metadata
- Vector similarity search on remaining candidates
- Distance < 0.25 → known cat, update profile
- Distance >= 0.25 → new cat, create profile with UUID
- No cats pass hard filter → automatically new cat

5. Agent Loop — agent/loop.py

The reasoning core. Implements a tool-use loop where the model decides what to do, we execute it, feed the result back, and repeat until the model signals it is done.

Tools available to the agent:

<code>extract_best_frames(video_path)</code>	Extract and score frames from a clip. Always called first.
<code>analyze_cat_image(image_path)</code>	Analyze a single frame. Returns structured cat description.
<code>process_visit(analysis, clip_path)</code>	Log visit to database. Returns known/new cat status.

Typical iteration sequence:

- Iteration 1 → agent calls `extract_best_frames()`
- Iterations 2-5 → agent calls `analyze_cat_image()` on each frame
- Iteration 6 → agent calls `process_visit()` with consolidated analysis
- Iteration 7 → agent writes summary and stops

The agent decides this sequence on its own. We never tell it the order — only describe what each tool does and set a goal.

Safety limit:

`max_iterations = 10` prevents runaway loops. If the agent hasn't finished in 10 iterations something has gone wrong.

6. Folder Watcher — watcher.py

Monitors the FTP drop folder for new video files. When a new clip appears it waits for the upload to complete (file size stable for 3 seconds) then triggers the agent loop.

Run:

```
python3 watcher.py /mnt/newdrive/srv/files/lilbit/uploads
```

Runs indefinitely. Logs all activity to `catmonitor.log` and `stdout`.

7. Batch Processor — process_existing.py

Processes clips already in the FTP folder. Separate from the watcher — each script has one job.

```
# See what's there without processing
python3 process_existing.py /path/to/uploads --dry-run

# Process only a specific camera and date
python3 process_existing.py /path/to/uploads \
  --camera "Platform Right" \
  --date "20260327"

# Process everything
python3 process_existing.py /path/to/uploads
```

Known Issues & Decisions

Plastic trigger	Piece of plastic moving in wind triggers motion sensor. Fix in Reolink app — draw motion zone excluding the plastic area.
Clip length	Default 30s clips. Reduce to 15s in Reolink — sufficient for cat identification, halves processing time.
Frame sampling	Currently fixed 1fps. Consider TARGET_FRAMES=20 approach so sampling adapts to clip length.
Eye color	Platform Right is overhead — cat eating with head down means eye color often unknown. Platform Front face-on shot captures it.
Tabby stripes	Front camera + bright light washes out stripe detail. Model calls orange tabby 'solid'. Platform Right overhead angle catches it better.
FTP permissions	Manually uploaded files may have wrong permissions. Fix: sudo chown -R walterp:walterp /uploads/. Reolink-uploaded files are fine.
GPU memory fault	ROCm throws memory fault on unclean Docker stop. Fix: docker stop \$(docker ps -q) then check rocm-smi before restarting.
Context window	max-model-len 16384. Full 4K frames at 8178 tokens fill this instantly — always resize to 640px wide before sending.

Planned — Next Steps

Reflective Agent

A second agent mode that runs during idle periods and reasons over visit history rather than individual clips. Needs at least one week of real data to be meaningful.

New tools needed:

- `get_cat_history(cat_id, limit)` — recent visits for a known cat
- `get_absent_cats(days)` — cats not seen in N days
- `get_health_trends(cat_id)` — body condition across visits
- `get_overnight_summary()` — all visits in last 12 hours

What it will detect:

- Body condition declining across visits
- Cats that have stopped visiting — flag absence after N days
- Changes in visit frequency or timing
- Health flags appearing for the first time
- New cats that visited once and never returned

Vector Database — Second Use Case

ChromaDB is currently used only for cat identity matching. A second collection could store visit descriptions as embeddings, enabling semantic search: 'find all visits where the cat seemed distressed' without needing exact field matches.

Self-Extending Agent

The agent recognizes it lacks a capability, writes the tool to fill the gap using Claude Code, tests it, and adds it to its own toolkit. Requires the base agent loop to be solid first. The architecture already supports it — Claude Code is available locally and the agent loop is modular.

Dashboard

Local web interface (Flask or FastAPI) showing visit timeline, cat roster, health notes, and alert inbox. Accessible from any device on the local network. Clips link to raw footage on the SATA/external drives.

What This Project Teaches About Agents

An agent is: LLM + Tools + Loop

Nothing more. The LLM reasons. Tools are functions it can call. The loop continues until the LLM decides it is done. Everything else is infrastructure around those three things.

The difference between a logger and an autonomous monitor

A logger records what happened when you tell it to. An autonomous monitor decides what to look at, reasons about what it finds, compares it against history, and tells you what matters. The cat monitor starts as the former and is being built toward the latter.

Structured output matters

Free text responses from the vision model cannot be stored, compared, or reasoned about programmatically. Structured JSON with controlled vocabularies — poor/fair/good/excellent rather than prose descriptions — is what makes the data useful across visits.

Hard filters beat soft similarity for identity

A general-purpose embedding model treats 'orange' and 'black' as similar because they are both colors. Adding hard metadata filters on coat_color and eye_color before running vector similarity search correctly separates obviously different cats without tuning thresholds.

Local inference changes the architecture

A cloud API handles concurrency for you. A single local vLLM instance processes one request at a time. Sending 120 frames concurrently to a local model causes queue overflow and errors. Sequential processing with MAX_CONCURRENT=1 is the correct approach for a single GPU.

Start simple, add on later — no issues

Every layer added cleanly without breaking what came before. The folder watcher does not know about ChromaDB. The identity engine does not know about the agent loop. The agent loop does not know about FFmpeg. Clean separation means each piece can be replaced or extended independently.